

Artificial Intelligence Practical Handbook

Table of Contents

1. [Experiment 1: Depth First Search \(DFS\) and Breadth First Search \(BFS\)](#)
 2. [Experiment 2: A* \(A-Star\) Search Algorithm](#)
 3. [Experiment 3: Greedy Search Algorithms](#)
 - o 3.1 [Selection Sort](#)
 - o 3.2 [Dijkstra's Shortest Path Algorithm](#)
 - o 3.3 [Prim's Minimum Spanning Tree Algorithm](#)
 - o 3.4 [Kruskal's Minimum Spanning Tree Algorithm](#)
 - o 3.5 [Job Scheduling Problem](#)
 4. [Experiment 4: N-Queens Problem \(Backtracking\)](#)
 5. [Experiment 5: Elementary Chatbot for Customer Interaction](#)
-

Experiment 1: Depth First Search (DFS) and Breadth First Search (BFS)

1. Theory

Introduction

Graph traversal is the process of visiting all nodes (vertices) in a graph systematically. In Artificial Intelligence, traversal algorithms are fundamental for searching state spaces, finding paths, and solving puzzles.

Definitions

- **Graph:** A non-linear data structure consisting of Vertices (Nodes) and Edges (Connections).
- **Undirected Graph:** A graph where edges have no direction (e.g., if A is connected to B, B is also connected to A).
- **DFS:** An algorithm that explores as deep as possible along each branch before backtracking.
- **BFS:** An algorithm that explores level by level, visiting all neighbors of a node before

moving to the next depth.

Core Concepts

- **Graph Representation:** We use an **Adjacency List**, where every vertex stores a list of its neighbors. This is memory-efficient for sparse graphs.
- **Data Structures:**
 - DFS uses a **Stack** (often implemented via recursion).
 - BFS uses a **Queue** (FIFO - First In First Out).
- **Visited Set:** Essential to track nodes already processed to prevent infinite loops in cyclic graphs.

Complexity and Characteristics

Feature	DFS	BFS
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$ (Stack depth)	$O(V)$ (Queue size)
Shortest Path	No	Yes (for unweighted)
Completeness	No (can fail in infinite paths)	Yes
Optimality	No	Yes

2. Algorithm (Exam-Writing Style)

Depth First Search (Recursive)

1. **Start**
2. Create a **visited** set to keep track of explored nodes.
3. Define a function DFS(vertex):
 - Mark the current vertex as **visited**.
 - **Print** the vertex.
 - For every **neighbor** of the current vertex:
 - If the neighbor is **not in visited**, recursively call DFS(neighbor).
4. **Stop** when all reachable nodes are processed.

Breadth First Search

1. **Start**
2. Create a **visited** set and a **queue**.
3. Add the starting vertex to the queue and mark it as **visited**.
4. **While** the queue is not empty:
 - **Pop** the front vertex from the queue.
 - **Print** the vertex.
 - For every **neighbor** of the popped vertex:
 - If the neighbor is **not visited**:
 - Mark it as **visited**.
 - **Push** it into the queue.

5. Stop

3. Full Code (Python)

Python

```
from collections import deque

def dfs(vertex, graph, visited):
    visited.add(vertex)
    print("Visited : ", vertex)

    for neighbour in graph[vertex]:
        if neighbour not in visited:
            dfs(neighbour, graph, visited)

def bfs(start, graph):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print("Visited : ", vertex)

        for neighbour in graph[vertex]:
            if neighbour not in visited:
                queue.append(neighbour)
                visited.add(neighbour)

# Graph (Adjacency List)
graph = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}
```

```
print("DFS Traversal : \n")
dfs(0, graph, set())
```

```
print("\nBFS Traversal : \n")
bfs(0, graph)
```

4. Line-by-Line Explanation

DFS Function

- `def dfs(vertex, graph, visited)::` Defines the function taking the current node, the graph structure, and the set of visited nodes.
- `visited.add(vertex):` Marks the current node so the algorithm doesn't process it again.
- `for neighbour in graph[vertex]:` Iterates through the list of nodes connected to the current vertex.
- `if neighbour not in visited::` Check to prevent cycles and redundant work.
- `dfs(neighbour, graph, visited):` The recursive step; it dives deeper into the graph.

BFS Function

- `queue = deque([start]):` Initializes a double-ended queue with the starting node. Deque is used for $O(1)$ pop operations.
- `while queue::` Continues the loop as long as there are nodes "discovered" but not yet "processed."
- `vertex = queue.popleft():` Removes the oldest element added to the queue (FIFO).
- `queue.append(neighbour):` Adds new neighbors to the back of the queue to be visited later.

5. Execution Trace (Dry Run)

Graph Input: 0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]

DFS Trace (Starting at 0)

1. Visit **0**. Neighbors: [1, 2].
2. Move to **1**. Neighbors: [0, 3]. (0 is visited).
3. Move to **3**. Neighbors: [1, 2]. (1 is visited).
4. Move to **2**. Neighbors: [0, 3]. (0, 3 are visited).
5. **Output:** 0 -> 1 -> 3 -> 2

BFS Trace (Starting at 0)

1. Queue: [0]. Pop **0**. Neighbors [1, 2] added to Queue.
2. Queue: [1, 2]. Pop **1**. Neighbor [3] added to Queue.
3. Queue: [2, 3]. Pop **2**. Neighbor [3] already in queue/visited.
4. Queue: [3]. Pop **3**.
5. **Output:** 0 -> 1 -> 2 -> 3

6. Viva Questions and Answers (40+)

1. **What is a Graph?** A collection of vertices and edges.
2. **What is the difference between a Tree and a Graph?** A tree is a connected graph without cycles.
3. **Why use an Adjacency List over an Adjacency Matrix?** List is $O(V+E)$ space, Matrix is $O(V^2)$. Lists are better for sparse graphs.
4. **What is the role of a Stack in DFS?** It stores the path for backtracking.
5. **Which algorithm finds the shortest path?** BFS (in unweighted graphs).
6. **What is Backtracking?** Returning to a previous node to explore other branches.
7. **What happens if you don't use a Visited set?** The algorithm enters an infinite loop in cyclic graphs.
8. **Time complexity of BFS/DFS?** $O(V + E)$.
9. **Space complexity of BFS?** $O(V)$ because of the queue.
10. **Is DFS recursive?** It can be both recursive or iterative (using an explicit stack).

11. **Application of DFS?** Cycle detection, topological sorting, solving mazes.
12. **Application of BFS?** Shortest path in GPS, social network levels, web crawling.
13. **What is FIFO?** First In First Out (used in Queues/BFS).
14. **What is LIFO?** Last In First Out (used in Stacks/DFS).
15. **Why is BFS memory-heavy?** It stores all nodes of the current level.
16. **Can BFS be recursive?** Possible but complex and rarely used.
17. **What is a directed graph?** Edges have a specific direction ($A \rightarrow B \neq B \rightarrow A$).
18. **What is a connected component?** A subgraph where all nodes are reachable from each other.
19. **What is 'deque' in Python?** A double-ended queue that allows fast appends and pops.
20. **Why initialize DFS visited set outside the recursive function?** To maintain the state across all recursive calls.
21. **How do you handle disconnected graphs in BFS?** Run BFS starting from every unvisited node.
22. **What is the branching factor?** The average number of successors for nodes in the graph.
23. **Is BFS optimal?** Yes, it always finds the shallowest goal.
24. **Is DFS complete?** No, it can get lost in infinite paths.
25. **Real-life example of BFS?** Finding friends of friends on Facebook.
26. **Real-life example of DFS?** Solving a Sudoku puzzle.
27. **What is a cycle?** A path where the start and end nodes are the same.
28. **Difference between Adjacency List and Adjacency Matrix?** List stores neighbors; Matrix stores 0/1 for every possible connection.
29. **What is level-order traversal?** Another name for BFS.
30. **What is the maximum number of edges in a graph with V vertices?** $V(V-1)/2$ for

undirected.

- 31. **Can DFS find the shortest path?** Only by chance; it doesn't guarantee it.
- 32. **Why is recursion simplified in DFS?** Python manages the call stack automatically.
- 33. **What is a weight in a graph?** A value assigned to an edge (cost, distance).
- 34. **What is a dense graph?** A graph where edges approach V^2 .
- 35. **What is a sparse graph?** A graph with very few edges.
- 36. **What is topological sort?** Ordering of nodes in a DAG based on dependencies.
- 37. **Can BFS work on weighted graphs?** No, it treats all edges as having weight 1.
- 38. **Which algorithm uses more memory?** BFS usually, due to the queue width.
- 39. **What is 'popleft'?** A method in Python's deque to remove the first element.
- 40. **How to define an undirected edge in a dictionary?** Add B to A's list and A to B's list.

Experiment 2: A* (A-Star) Search Algorithm

1. Theory

Introduction

A* Search is one of the most powerful and popular algorithms in AI for pathfinding and graph traversal. It is an **informed search** algorithm because it uses "heuristics" to guide its search.

Core Concept: The Formula

A* uses a evaluation function $f(n)$ to decide which node to expand next:

$$f(n) = g(n) + h(n)$$

- **$g(n)$** : The actual cost to reach node n from the start node.
- **$h(n)$** : The heuristic estimated cost to get from node n to the goal.
- **$f(n)$** : Total estimated cost of the path through n .

The 8-Puzzle Problem

In this experiment, we solve the 8-puzzle. The goal is to move tiles (1-8) in a 3×3 grid using a blank space (0) to reach a target configuration.

Heuristic Function

We use the **Misplaced Tiles Heuristic**: It counts the number of tiles that are not in their correct goal position.

Complexity and Characteristics

- **Time Complexity**: $O(b^d)$, where b is the branching factor and d is the depth.
- **Space Complexity**: $O(b^d)$, as it stores all generated nodes in memory.
- **Completeness**: Yes, it will find a solution if one exists.
- **Optimality**: Yes, provided the heuristic is **admissible** (never overestimates the cost).

2. Algorithm (Exam-Writing Style)

1. **Start**
2. Create an **OPEN** list (Priority Queue) and a **CLOSED** list (Set).
3. Calculate $h(\text{start})$ and $f(\text{start}) = g(\text{start}) + h(\text{start})$. (Note: $g(\text{start}) = 0$).
4. Push the starting state into the **OPEN** list.
5. **Repeat** until OPEN is empty:
 - **Pop** the node with the **lowest $f(n)$** from OPEN.

- If this is the **Goal State**, stop and print success.
 - Add the current node to the **CLOSED** list.
 - **Generate all possible moves** (Up, Down, Left, Right) for the blank tile.
 - For each new child state:
 - If child is in CLOSED, skip it.
 - Calculate $g(\text{child}) = g(\text{parent}) + 1$.
 - Calculate $h(\text{child})$ and $f(\text{child}) = g(\text{child}) + h(\text{child})$.
 - Push the child into the **OPEN** list.
6. If no solution found, print failure.
7. **Stop**

3. Full Code (Python)

Python

```
import heapq

N = 3
goal = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 0]]

def calculate_heuristic(mat):
    count = 0
    for i in range(N):
        for j in range(N):
            if mat[i][j] != 0 and mat[i][j] != goal[i][j]:
                count += 1
    return count
```

```

def is_goal(mat):
    return mat == goal

def solve_astar(start, x, y):
    visited = set()
    pq = []
    h = calculate_heuristic(start)
    # PQ stores: (f_score, g_score, matrix, blank_x, blank_y)
    heapq.heappush(pq, (h, 0, start, x, y))

    row = [1, 0, -1, 0] # Down, Right, Up, Left
    col = [0, 1, 0, -1]

    while pq:
        f, cost, mat, x, y = heapq.heappop(pq)
        mat_tuple = tuple(tuple(r) for r in mat)

        if mat_tuple in visited:
            continue
        visited.add(mat_tuple)

        if is_goal(mat):
            print(f'Goal state reached successfully with cost : {cost}')
            return

        for i in range(4):
            newX, newY = x + row[i], y + col[i]
            if 0 <= newX < N and 0 <= newY < N:
                new_mat = [r[:] for r in mat]
                new_mat[x][y], new_mat[newX][newY] = new_mat[newX][newY], new_mat[x][y]
                new_tuple = tuple(tuple(r) for r in new_mat)

                if new_tuple not in visited:
                    h = calculate_heuristic(new_mat)
                    g = cost + 1
                    heapq.heappush(pq, (g + h, g, new_mat, newX, newY))

    print("Solution not found !")

# Initial puzzle configuration
start = [[1, 2, 3],
         [5, 6, 0],

```

```
[7, 8, 4]]
```

```
solve_astar(start, 1, 2)
```

4. Line-by-Line Explanation

- `import heapq`: Imports the priority queue library required to always pick the smallest $f(n)$.
- `calculate_heuristic(mat)`: Compares current board tiles to the goal board. Ignores the blank tile (0).
- `mat_tuple = tuple(tuple(r) for r in mat)`: Converts the list matrix to a nested tuple because lists are mutable and cannot be added to a Python set.
- `heapq.heappush(pq, (g+h, g, new_mat, newX, newY))`: Adds the state to the priority queue. The first element of the tuple ($g+h$) determines the priority.
- `new_mat = [r[:] for r in mat]`: Creates a **deep copy** of the matrix. If we don't do this, changing `new_mat` would also change the parent `mat`.

5. Viva Questions and Answers (40+)

1. **What is Informed Search?** Search that uses domain-specific knowledge (heuristics) to find solutions more efficiently.
2. *Explain the A formula.* * $f(n) = g(n) + h(n)$. g is actual cost, h is estimated cost to goal.
3. **What is a Heuristic?** An "educated guess" or estimate used to speed up searching.
4. **What is an Admissible Heuristic?** One that never overestimates the actual cost to reach the goal.
5. **What is the Manhattan Distance heuristic?** The sum of absolute differences of x and y coordinates of tiles from their goal positions.
6. *Why use a Priority Queue in A?* * To efficiently retrieve the node with the minimum $f(n)$ value.

7. *Is A optimal?** Yes, if the heuristic is admissible.
8. *Difference between BFS and A?** BFS is uninformed (heuristic $h(n)=0$); A* is informed.
9. *Difference between Greedy Best First Search and A?** Greedy only uses $h(n)$; A* uses both $g(n)$ and $h(n)$.
10. **What happens if $h(n)=0$ always?** A* becomes Uniform Cost Search (like Dijkstra).
11. **What is the branching factor of the 8-puzzle?** Between 2 and 4 (depending on where the blank tile is).
12. **What is the state space of 8-puzzle?** $9!$ possible arrangements (362,880).
13. **Why do we need the 'visited' set?** To avoid "cycles" or re-processing the same board configuration multiple times.
14. **What is the time complexity?** Exponential, $O(b^d)$.
15. *What is a 'node' in A?** A specific state of the board.
16. *Why is A called 'optimal'?** Because it is guaranteed to find the path with the lowest cost.
17. *What is the disadvantage of A?** It uses a lot of memory because it stores all generated nodes.
18. *Can A work on real-world maps?** Yes, it is the standard for GPS navigation.
19. **What if the heuristic is not admissible?** A* might find a solution, but it is no longer guaranteed to be the shortest/optimal one.
20. **What is the blank tile represented as?** Zero (0).
21. **What are moves allowed in the 8-puzzle?** Up, Down, Left, Right.
22. **What is the 'OPEN' list?** Nodes that have been discovered but not yet expanded.
23. **What is the 'CLOSED' list?** Nodes that have already been expanded.
24. *Why is A better than DFS?** DFS can go down wrong paths infinitely; A* uses $f(n)$ to stay on the most promising path.
25. **Is the misplaced tile heuristic admissible?** Yes, because each misplaced tile must be

moved at least once.

26. **What is the size of the 8-puzzle board?** 3×3 .
27. **What is the goal state?** The target arrangement of tiles.
28. **How many tiles are in an 8-puzzle?** 8 numbered tiles and 1 blank.
29. **What is 'heapq.heappop'?** A function that removes and returns the smallest item from the heap.
30. **What is 'tuple(tuple(r) for r in mat)'?** Nested list comprehension to create an immutable version of the board.
31. **What is the branching factor 'b'?** Number of children per node.
32. **What is depth 'd'?** Length of the path to the goal.
33. *Can A solve a 15-puzzle?* Yes, but the state space is much larger ($16!$).
34. **What is an 'informed' search?** A search using a heuristic.
35. **What is pathfinding?** Finding the shortest sequence of steps to a goal.
36. **What is state expansion?** Generating all possible next states from a current state.
37. **What happens if the goal is unreachable?** The algorithm will eventually empty the OPEN list and return "Solution not found."
38. **Why is $g(n)$ important?** It keeps the algorithm "honest" by accounting for the work already done.
39. **Why is $h(n)$ important?** It directs the algorithm toward the goal.
40. *Name one game that uses A.** Most strategy games (like StarCraft) or pathfinding in RPGs.

Experiment 3: Greedy Search Algorithms

3.1 Selection Sort

1. Theory

Selection Sort is a greedy comparison-based sorting algorithm. It is "Greedy" because at each step, it chooses the **local optimum** (the smallest element in the remaining list) and places it in the sorted section.

- **Logic:** Divide the array into sorted and unsorted parts. Repeatedly find the minimum from the unsorted part and swap it with the first unsorted element.
- **Time Complexity:** $O(n^2)$ for all cases (Best, Average, Worst).
- **Space Complexity:** $O(1)$ (In-place sorting).

2. Algorithm

1. **Start**
2. Iterate i from 0 to $n-1$.
3. Set $\text{min_index} = i$.
4. Iterate j from $i+1$ to n .
5. If $\text{arr}[j] < \text{arr}[\text{min_index}]$, update $\text{min_index} = j$.
6. **Swap** $\text{arr}[i]$ with $\text{arr}[\text{min_index}]$.
7. **Repeat** until the list is sorted.
8. **Stop**

3. Code & Explanation

Python

```
arr = [64, 25, 12, 22, 11]
```

```
for i in range(len(arr)):
```

```

min_index = i
for j in range(i+1, len(arr)):
    if arr[j] < arr[min_index]:
        min_index = j
arr[i], arr[min_index] = arr[min_index], arr[i]

```

- `min_index = i`: We assume the current element is the smallest.
- `arr[i], arr[min_index] = ...`: This performs the **swap** in a single line using Python's tuple unpacking.

4. Viva Questions (Top 5)

1. **Why is Selection Sort greedy?** It picks the smallest element immediately.
2. **Is it stable?** No, swapping can change the relative order of equal elements.
3. **Why is the best case $O(n^2)$?** Because it still performs all comparisons even if sorted.
4. **How many swaps are performed?** At most $n-1$ swaps.
5. **What is 'in-place'?** Sorting without requiring additional memory.

3.2 Dijkstra's Shortest Path Algorithm

1. Theory

Dijkstra's is a greedy algorithm used to find the shortest path from a single source node to all other nodes in a **weighted graph**.

- **Greedy Choice**: At each step, it picks the unvisited node with the smallest distance from the source.
- **Constraint**: Works only for non-negative edge weights.
- **Time Complexity**: $O(V^2)$ with arrays; $O((V+E) \log V)$ with Priority Queues.

2. Algorithm

1. **Start**

2. Set distance to source = 0, and all other nodes = Infinity.
3. Add source to Priority Queue.
4. While Queue is not empty:
 - Pop node u with minimum distance.
 - For every neighbor v of u :
 - $\text{new_dist} = \text{dist}[u] + \text{weight}(u, v)$.
 - If $\text{new_dist} < \text{dist}[v]$:
 - Update $\text{dist}[v] = \text{new_dist}$.
 - Push v into Priority Queue.
5. **Stop**

3. Code (Python)

Python

```
import heapq

def dijkstras(graph, start):
    distances = {node: float("inf") for node in graph}
    distances[start] = 0
    pq = [(0, start)]

    while pq:
        curr_dist, curr_node = heapq.heappop(pq)

        if curr_dist > distances[curr_node]: continue

        for neighbor, weight in graph[curr_node].items():
            dist = curr_dist + weight
```

```
if dist < distances[neighbor]:
    distances[neighbor] = dist
    heapq.heappush(pq, (dist, neighbor))
return distances
```

3.3 Prim's Minimum Spanning Tree Algorithm

1. Theory

Prim's algorithm finds the Minimum Spanning Tree (MST) of a weighted undirected graph. An MST is a subset of edges that connects all vertices with minimum total weight and no cycles.

- **Greedy Choice:** It grows the tree by adding the cheapest edge from a visited vertex to an unvisited vertex.
- **Data Structure:** Uses a Priority Queue to pick the minimum weight edge.

2. Algorithm

1. **Start**
2. Initialize a tree with a single starting vertex.
3. While MST does not include all vertices:
 - Find the edge with the **minimum weight** that connects a vertex in the tree to one outside the tree.
 - Add this edge and the new vertex to the tree.
4. **Stop**

3. Code (Python)

Python

```

import heapq

def prim(graph, start):
    visited = set()
    min_heap = [(0, start)]
    total_cost = 0

    while min_heap:
        weight, u = heapq.heappop(min_heap)
        if u in visited: continue

        visited.add(u)
        total_cost += weight
        print(f"Added vertex {u} with weight {weight}")

        for v, w in graph[u]:
            if v not in visited:
                heapq.heappush(min_heap, (w, v))
    print("Total MST Cost:", total_cost)

```

3.4 Kruskal's Minimum Spanning Tree Algorithm

1. Theory

Kruskal's is an **edge-based** greedy algorithm for finding the MST.

- **Logic:** Sort all edges in non-decreasing order of weight. Pick the smallest edge. If adding it doesn't form a cycle, add it to the MST.
- **Cycle Detection:** Uses the **Union-Find** data structure to check if two vertices are in the same component.

2. Algorithm

1. **Start**
2. **Sort** all edges by weight.
3. Create a separate set for each vertex.

4. For each edge (u, v) in sorted list:
 - If Find(u) is not equal to Find(v) (no cycle):
 - Add edge to MST.
 - Perform Union(u, v).
5. Stop

3. Code (Python)

Python

```
class Graph:
    def find(self, parent, i):
        if parent[i] == i: return i
        return self.find(parent, parent[i])

    def union(self, parent, rank, x, y):
        xr, yr = self.find(parent, x), self.find(parent, y)
        if rank[xr] < rank[yr]: parent[xr] = yr
        elif rank[xr] > rank[yr]: parent[yr] = xr
        else:
            parent[yr] = xr
            rank[xr] += 1
```

3.5 Job Scheduling Problem

1. Theory

The Job Scheduling problem aims to maximize profit by selecting jobs that can be completed within their deadlines. Each job takes 1 unit of time.

- **Greedy Choice:** Sort jobs in descending order of profit and assign them to the latest possible available time slot before their deadline.

2. Algorithm

1. **Sort** jobs by profit (high to low).
2. Find the **maximum deadline** to determine the number of slots.
3. For each job:
 - Check slots from deadline-1 down to 0.
 - If a slot is empty, assign the job and add profit.
4. Print results.

3. Code (Python)

Python

```
def job_scheduling(jobs):
    jobs.sort(key=lambda x: x[2], reverse=True)
    max_deadline = max(j[1] for j in jobs)
    slots = [-1] * max_deadline
    total_p = 0

    for jid, dl, profit in jobs:
        for i in range(dl - 1, -1, -1):
            if slots[i] == -1:
                slots[i] = jid
                total_p += profit
                break
    return total_p
```

3.6 Viva Questions for Experiment 3 (60+)

1. **What is a Greedy algorithm?** One that makes the best local choice at each step.
2. **Does Dijkstra work with negative weights?** No, because it assumes a path can't get shorter by adding edges.
3. **Difference between Prim's and Kruskal's?** Prim's is vertex-based; Kruskal's is edge-based.
4. **What is a Spanning Tree?** A subgraph with all vertices and no cycles.

5. **Complexity of Kruskal's?** $O(E \log E)$ due to sorting.
 6. **What is Union-Find?** A data structure that tracks elements split into disjoint sets.
 7. **What is relaxation in Dijkstra?** Updating a node's distance if a shorter path is found.
 8. **Why sort jobs in Job Scheduling?** To prioritize high-profit jobs (greedy approach).
 9. **What is the 'Rank' in Union-Find?** A way to keep the tree flat for efficiency.
 10. **Can Prim's work on disconnected graphs?** No, it only builds one component.
-

Experiment 4: N-Queens Problem (Backtracking)

1. Theory

Introduction

The N-Queens problem is a classic **Constraint Satisfaction Problem (CSP)**. The goal is to place N queens on an $N \times N$ chessboard such that no two queens attack each other.

Constraints

- No two queens in the same **Row**.
- No two queens in the same **Column**.
- No two queens on the same **Diagonal**.

Backtracking Approach

Backtracking is a recursive technique that tries a solution and, if it fails, "undoes" the last step and tries another option. It's essentially a Depth First Search on the state space tree.

- **Time Complexity:** $O(N!)$ (Worst Case).
- **Space Complexity:** $O(N^2)$ for the board.

2. Algorithm

1. **Start** at the first row.
2. For each **column** in the current row:
 - Check if the position is **Safe** (no other queens attack).
 - If **Safe**:
 - Place the queen.
 - Recursively call the function for the **next row**.
 - If the recursive call leads to a solution, return success.
 - **Backtrack**: Remove the queen (set slot to empty) and try the next column.
3. If all columns are tried and none work, return failure.
4. **Stop**

3. Code (Python)

Python

N = 4

```
def is_safe(board, row, col):
    # Check column
    for i in range(row):
        if board[i][col] == 'Q': return False

    # Check upper-left diagonal
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 'Q': return False
        i -= 1; j -= 1
```

```

# Check upper-right diagonal
i, j = row, col
while i >= 0 and j < N:
    if board[i][j] == 'Q': return False
    i -= 1; j += 1
return True

def solve(board, row):
    if row == N:
        for r in board: print(" ".join(r))
        print(); return True

    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 'Q'
            if solve(board, row + 1): return True # Stop at first solution
            board[row][col] = '.' # Backtrack
    return False

board = [['.' for _ in range(N)] for _ in range(N)]
solve(board, 0)

```

4. Viva Questions (40+)

1. **What is a Constraint Satisfaction Problem?** A problem defined by a set of variables and rules they must follow.
2. **Why is it called 'Backtracking'?** Because we go back to a previous state when we hit a "dead end."
3. **Is N-Queens possible for $N=2$ or $N=3$?** No, there are no valid solutions.
4. **How many solutions for $N=8$?** 92.
5. **What is pruning?** Skipping branches in the search tree that are guaranteed to fail.
6. **Why only check upper diagonals?** Because queens are placed row by row, so lower rows are empty.
7. **What is the state space?** The set of all possible ways to place N queens.

8. **Is Backtracking brute force?** It's a "refined" brute force that prunes paths.
9. **Difference between Backtracking and Branch & Bound?** Backtracking uses DFS; Branch & Bound uses BFS/Best-First and cost functions.

Experiment 5: Elementary Chatbot for Customer Interaction

1. Theory

Introduction

A chatbot is a software designed to simulate human conversation. This experiment focuses on a **Rule-Based Chatbot**, which follows a set of "if-then" conditions to respond to specific user keywords.

Core Concepts

- **NLP (Natural Language Processing):** The field of AI that allows computers to understand and process human language.
- **Keyword Matching:** The bot scans user input for specific words (e.g., "help", "hi") to trigger predefined responses.
- **Preprocessing:** Converting input to lowercase (.lower()) to ensure "Hi", "HI", and "hi" are treated the same.

Complexity

- **Time Complexity:** $O(n)$, where n is the number of rules/conditions to check.
- **Space Complexity:** $O(1)$ (minimal storage needed for predefined strings).

2. Algorithm

1. **Start**
2. Display a welcome message and a list of supported questions.

3. Enter an infinite while loop.
 4. **Take input** from the user.
 5. **Clean input:** Convert the string to lowercase.
 6. **Compare:**
 - If input matches "hi", print "Hello".
 - If input matches "name", print the bot's name.
 - If input matches "quit", break the loop.
 - **Else:** Print a default "I don't understand" message.
 7. **Stop**
-

3. Code (Python)

Python

```
print("Simple Question and Answering Program")
print("=====")

print("\nYou can ask the following questions:")
print("1. Hi\n2. How are you?\n3. What is your name?\n4. Quit\n")

while True:
    question = input("Enter your question: ").lower()

    if question == "hi":
        print("Hello")
    elif "how are you" in question:
        print("I am fine")
    elif "name" in question:
        print("My name is Emilia")
        name = input("Enter your name: ")
```

```
print("Nice meeting you", name)
elif question == "quit":
    print("Goodbye!")
    break
else:
    print("I don't understand. Please choose from the given questions.")
```

4. Viva Questions (40+)

1. **What is a Chatbot?** Software that simulates human conversation.
2. **Difference between AI and Rule-Based bots?** AI bots learn from data; Rule-based bots use fixed logic.
3. **Why use .lower()?** To make the matching case-insensitive.
4. **What is the 'else' block's purpose?** To handle unexpected user inputs (fallback).
5. **Name some famous chatbots.** Siri, Alexa, ChatGPT, Google Assistant.
6. **What is Intent Recognition?** Identifying the "goal" behind a user's message.
7. **Can a rule-based bot handle complex tasks?** No, it is limited to the programmed rules.
8. **What is a Virtual Assistant?** A more complex chatbot that can perform tasks (set alarms, play music).
9. **Application of chatbots in healthcare?** Symptom checkers and appointment booking.
10. **Why is infinite loop while True used?** To keep the bot running until the user says "quit".

Sources

1. <https://github.com/BijonDurjoy/AI>
2. <https://github.com/BijonDurjoy/AI>